concept: *graphemes*. Graphemes are simply symbols that represent the smallest unit of language in a writing system, and they can be single images or sequences of letters that represent a single sound. PHP's `intl` extension offers some grapheme-aware functions; here's a quick example (you'll need the `intl` PHP module installed, natch):

```
var_dump(bin2hex($prideFlag));
var_dump(grapheme_strlen($prideFlag));
```

```
Output:
string(28) "f09f8fb3efb88fe2808df09f8c88"
int(1)
```

We can see that our flag is represented by fourteen bytes, which constitute four UTF-8 characters (one for each codepoint we used): *0xF0 0x9F 0x8F 0xB3 + 0xEF 0xB8 0x8F + 0xE2 0x80 0x8D + 0xF0 0x9F 0x8C 0x88*. `mb_strlen()` is UTF-8 aware and dutifully reports *4* characters, but `grapheme_strlen()` is also aware of multi-codepoint encoding and returns *1*. Here's our bald man causing the same problems for us:

```
$baldMan = "\u{1F468}\u{200D}\u{1F9B2}";
var_dump(mb_strlen($baldMan));
var_dump(grapheme_strlen($baldMan));
```

```
Output:
int(3)
int(1)
```

substr_count(), mb_substr_count()

Moving on from basic counts of total length, how about counting occurrences of substrings? Here's a simple example using `substr_count()`:

```
$sayHello = 'hello hello hello';
var_dump(substr_count($sayHello, 'hello'));
var_dump(substr_count($sayHello, 'll'));
var_dump(substr_count($sayHello, 'l'));
```

```
Output:
int(3)
int(3)
int(6)
```